

PROJECT WALKTHROUGH

AML Command Center

How the Oracle JET frontend, gateway APIs, authentication, risk model, alerts, and investigations work together.

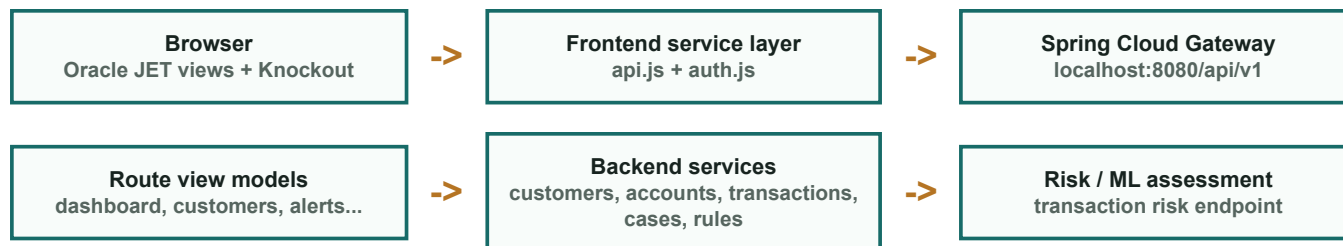
The one-sentence explanation

The browser is a Knockout MVVM dashboard that routes analysts between compliance modules, calls the Spring Cloud Gateway at <http://localhost:8080/api/v1>, enriches transactions with risk assessments, and turns model or rule matches into investigations.

Prepared for project explanation | 24 August 2026
Source reviewed: `src/js`, tests, configuration, and README

1. Architecture At A Glance

The application uses Oracle JET 16.1 with RequireJS, Knockout observables, and MVVM-style view models. The frontend does not retain a client-side database: modules load their data from services on demand.



Startup sequence

1. `src/js/main.js` configures RequireJS paths and loads root.
2. `root.js` waits for the document, then applies Knockout bindings to `#globalBody`.
3. `appController.js` configures routes, loads the active `oj-module`, and passes shared parameters such as the router, global search, and alert count.
4. The active view model calls `services/api.js`; its observables update the Oracle JET view when responses arrive.

Shared frontend responsibilities

appController

Navigation, authentication state, session user, mobile drawer, global search, and route-to-module wiring.

base.js

Common router access, money formatting, risk score normalization, risk category thresholds, status classes, and initials.

api.js

HTTP verbs, base URL, no-store caching, JSON parsing, response unwrapping, and consistent `ApiError` handling.

auth.js

Login request, token storage, bearer header injection, and token clearing. The active config enables JWT.

2. The User-Facing Modules

Route / module	Purpose	Main data behavior
Dashboard dashboard.js	Executive AML workspace: counts, recent activity, customer risk distribution, and transaction screening.	Loads customers, accounts, and transactions. Separately requests risk for each transaction.
Customer risk customers.js	Searchable customer list with KYC, account count, country-risk flag, and derived risk.	Loads customers plus accounts; calculates customer risk from linked account scores.
Customer detail customer-detail.js	One customer, linked accounts, related transactions, and risk explanations.	Loads the selected customer, all accounts, all transactions, then risk-enriches related transactions.
Transactions transactions.js	Searchable transaction monitoring view with flags, risk score, explanation, recommendation, and case link.	Loads transactions and cases, then calls the risk endpoint per transaction.
AML alerts alerts.js	Combines automatic model flags and active amount-threshold rule matches into one analyst queue.	Loads transactions, cases, rules, and risk per transaction; creates a case through POST /cases.
Investigations investigations.js	Case queue, owner assignment, escalation, resolution, analyst timeline, and report preview.	Loads cases; updates them with PUT /cases/{id}. Timeline notes are session-only.
Reports reports.js	Generates an investigation report or a workspace activity snapshot in the browser.	Reads cases plus either the selected customer or all transactions; downloads text content.
Detection rules rules.js	Create, activate/deactivate, delete, and review transaction amount rules.	Loads rules and transactions; calculates hits locally for the last 30 days.

Risk scoring detail: scores are normalized from either 0-1 or 0-100 to a percentage. Categories are ranked as UNASSESSED, LOW, MEDIUM, HIGH, CRITICAL. The UI uses the backend category label when comparing severity.

3. API Contract And Flow

All normal module calls go through `api.get`, `api.post`, `api.put`, or `api.remove`. The client prefixes each path with `http://localhost:8080/api/v1`.

Method	Path	Used for
POST	/auth/login	Authenticate and receive token or accessToken.
GET	/customers /customers/{id}	Customer list and customer detail.
GET	/accounts	Link accounts to customers and populate screening form options.
GET / POST	/transactions /transactions/{id}/risk	Read transactions, submit a transaction, and retrieve its AML assessment.
GET / POST / PUT	/cases /cases/{id}	Read cases, create an investigation from an alert, and update ownership/status.
GET / POST / PUT / DELETE	/rules /rules/{id}	Manage amount-threshold detection rules.

Standard request lifecycle

1. View model calls `api.get/post/put/remove(path, body)`
2. `api.js` adds the base URL, JSON content type for bodies, and Authorization: Bearer token when present
3. `fetch` sends cache: no-store to the gateway
4. `api.js` parses JSON, accepts 204 as null, and unwraps `response.data`
5. `success:false` or non-2xx becomes `ApiError(message, status, details)`
6. View model catches the error and exposes the message through its observable error state

Response shape expected by the client

```
{ "success": true, "data": [ ... ] }

{ "success": false, "message": "Invalid account" }
```

The API helper also accepts an unwrapped payload. A malformed JSON response is reported as an invalid gateway response.

4. End-To-End AML Scenario

This is the easiest story to tell in a demo: screen a transaction, assess risk, show the alert, and open an investigation.

1. INPUT

Dashboard screening form validates sender, receiver, amount, and date. It builds a transaction payload with currencies, bank locations, payment type, timestamp, and optional Gemini flag.

2. CREATE

POST /transactions persists or submits the transaction through the gateway.

3. ASSESS

GET /transactions/{id}/risk returns category, score, flag decision, confidence, explanation, and recommendation.

4. DISPLAY

Dashboard shows the result and reloads its customer, account, transaction, and risk-derived metrics.

5. MERGE

Alerts loads active rules and compares transaction amount to each threshold. Model flags and rule matches become one alert with source labels.

6. INVESTIGATE

Analyst assigns an owner or creates a case. The frontend sends POST /cases, then the investigation queue can update status through PUT /cases/{id}.

What is calculated in the browser

- Customer risk list values from linked account risk scores.
- Dashboard risk distribution and 14-day activity chart.
- Rule hit counts for transactions within the previous 30 days.
- Alert source merging, severity selection, filters, and search.
- Investigation report text and session-only analyst timeline notes.

Important boundary: the frontend displays and combines risk signals; it is not the ML model. The backend/risk service remains the source of the assessment returned by the risk endpoint.

5. Authentication, Runtime, And Caveats

JWT flow

1. Login validates non-empty credentials in `appController.js`.
2. `auth.login` sends them to `POST /api/v1/auth/login`.
3. The returned token is stored under `aml_access_token` in session storage, with a local-storage fallback in non-browser tests.
4. Every API request copies the caller headers and adds `Authorization: Bearer <token>`.
5. Logout clears the token and the demo session record.

Run and build

```
npm install installs dependencies
npm test runs API and auth unit tests
npm run serve starts the Oracle JET development server
npm run build creates the optimized application in web/
npm run verify runs tests followed by the release build
```

Configuration to know

- Node engine is 20.x.
- Gateway base URL is `http://localhost:8080/api/v1`.
- The gateway, Eureka, transaction service, and ML service need to be running for live data.
- The dev gateway must allow origins `http://localhost:8000` and `http://127.0.0.1:8000`.

Documentation mismatch: the current code has `auth.enabled: true` and performs backend login, but the README still says JWT is disabled and the login is local-only. For today's explanation, describe the code behavior as implemented: JWT is enabled and expected at `/auth/login`. Update the README later so setup instructions match the source.

Closing explanation

“The frontend is a route-driven compliance workspace. Each module owns its data loading and presentation, while the shared API service standardizes gateway communication and errors. Transactions are enriched with risk assessments, and the alerts module joins those model results with configurable rules before analysts manage the resulting cases.”

AML Command Center project guide | Generated from the current frontend source